

1) Introduction

Le déploiement d'algorithmes de vision par ordinateur de pointe sur des systèmes embarqués à ressources restreintes (Edge AI) constitue l'un des verrous technologiques majeurs de la robotique autonome, des véhicules intelligents et des dispositifs mobiles. Dans le cadre de la navigation, de la localisation et de la cartographie simultanées (SLAM) ou de l'odométrie visuelle, l'extraction et la mise en correspondance de points clés géométriques stables d'une image à l'autre représentent le socle fondamental de la perception spatiale.

Traditionnellement, ce traitement s'exécute en deux blocs asymétriques :

- **L'extracteur (Front-End) :** Chargé de détecter des points géométriques remarquables et de leur associer une signature numérique unique appelée descripteur.
- **Le matcher (Back-End) :** Chargé de résoudre le problème d'appariement géométrique et d'éliminer les fausses correspondances (outliers).

L'objectif central de ce projet est d'optimiser le pipeline de référence de l'état de l'art, composé de SuperPoint pour l'extraction et de LightGlue pour la mise en correspondance.

Ce rapport documente la démarche que j'ai menée au cours du semestre afin d'optimiser ce pipeline en utilisant plusieurs méthodes comme la quantification, la *pruning* ou encore la distillation de connaissances. Nous verrons que j'ai rencontré plusieurs difficultés et quelles méthodes j'ai utilisées afin de les résoudre.

2) Explication de SuperPoint

SuperPoint est un CNN conçu pour traiter des images en une seule passe, c'est-à-dire qu'il va simultanément extraire deux informations : la localisation des points d'intérêt et leurs descripteurs, alors qu'auparavant ces deux tâches étaient généralement séparées. Pour cela, il utilise un encodeur partagé afin d'éviter de recalculer deux fois les caractéristiques de l'image. Grâce à ce mécanisme, le modèle peut atteindre une vitesse d'environ 70 images par seconde. L'encodeur utilisé est de type VGG-like (il ne s'agit pas d'un VGG pré-entraîné). Il est composé de 8 couches de convolution organisées en paires de deux, avec une structure de canaux 64-64-64-64-128-128-128-128. Toutes les deux couches, une opération de max-pooling 2×2 est appliquée. Au

total, l'image est donc sous-échantillonnée par un facteur 8. Ainsi, après le passage dans l'encodeur, chaque pixel de cette carte compressée correspond à une cellule de 8×8 pixels de l'image d'origine. Le point où SuperPoint se démarque réellement est l'utilisation de deux décodeurs distincts : l'un dédié à la détection des points d'intérêt et l'autre à leur description. Pour le décodeur de points d'intérêt, le réseau ne produit pas directement une carte de la taille de l'image d'origine. Il génère un tenseur de taille $H_c \times W_c \times 65$ (où H_c et W_c correspondent aux dimensions de l'image divisées par 8 après l'encodeur). Les 64 premiers canaux correspondent aux positions possibles des points dans chaque cellule 8×8 , tandis que le 65^e canal, appelé dustbin, indique qu'aucun point d'intérêt n'est présent dans cette cellule. Pour retrouver la résolution originale, on applique d'abord un softmax sur les 65 canaux, puis on supprime le canal dustbin. Ensuite, une opération de sub-pixel convolution (ou pixel shuffle) est utilisée afin de réorganiser les valeurs et retrouver une carte de taille $H \times W$. Le décodeur de descripteurs, quant à lui, calcule un descripteur unique par cellule de 8×8 , ce qui correspond à une grille semi-dense. Ce choix permet de réduire la mémoire utilisée ainsi que le temps de calcul. Le réseau applique ensuite une interpolation bicubique pour obtenir une grille dense $H \times W$, cette méthode prenant en compte les 16 voisins les plus proches (contrairement à l'interpolation bilinéaire qui n'en considère que 4). Enfin, afin d'optimiser les comparaisons, une normalisation L2 est appliquée : chaque vecteur est divisé par sa norme euclidienne. Ainsi, la comparaison entre deux descripteurs peut se faire simplement par un produit scalaire, une opération très rapide sur un processeur.

Concernant l'entraînement, il se déroule en trois étapes cruciales. Premièrement, puisqu'il n'existe pas de base de données d'images réelles contenant des millions de points d'intérêt parfaitement annotés, les auteurs commencent par créer un environnement artificiel contrôlé. Ils génèrent un jeu de données de formes géométriques simples telles que des triangles, des quadrilatères, des cubes ou encore des étoiles. Dans ce contexte, la position exacte de chaque point d'intérêt est connue, ce qui fournit une ground truth fiable. Des déformations homographiques sont ensuite appliquées afin d'augmenter les données. À cette étape, seule la partie détecteur du réseau est utilisée (l'encodeur et la tête de détection). Ce sous-modèle est appelé MagicPoint et il apprend à détecter ces points d'intérêt synthétiques. Cependant, après cet entraînement, MagicPoint est très performant sur des formes simples mais beaucoup moins sur des images réelles. Pour pallier ce problème, les auteurs introduisent l'Homographic Adaptation. Le principe repose sur la cohérence géométrique : un bon détecteur doit être covariant, c'est-à-dire que lorsqu'une image est transformée, les points détectés doivent subir la même transformation.

Concrètement, on applique $N_h = 100$ homographies aléatoires (rotations, zooms, changements de perspective) à une image réelle. Chaque image transformée est passée dans MagicPoint, puis les résultats sont reprojetés dans l'espace de l'image originale et superposés. Si un point apparaît de manière cohérente dans la majorité des transformations, il est alors validé comme un "vrai" point d'intérêt. Le modèle génère

ainsi ses propres annotations, ce qui permet un entraînement auto-supervisé (self supervised). Grâce à ce procédé, on obtient un ensemble d'images réelles accompagnées de points d'intérêt pseudo-annotés, appelés pseudo-ground truth labels. Il devient alors possible d'entraîner le modèle complet, incluant à la fois la détection et la description. Pour cela, on prend une image I à laquelle on applique une homographie aléatoire H afin d'obtenir une image I' . La correspondance géométrique entre les pixels de I et de I' est donc parfaitement connue. Le réseau doit alors minimiser une perte totale L , qui est la somme de deux pertes. La perte de détection L_p repose sur une entropie croisée calculée sur les 65 canaux de chaque cellule 8×8 . La perte de descripteur L_d est une hinge loss avec une marge positive de 1 et une marge négative de 0,2. En termes d'hyperparamètres, un poids $\lambda = 0,0001$ est utilisé pour équilibrer les pertes de détection et de description. Un terme de pondération $\lambda_d = 250$ est également introduit dans la perte des descripteurs afin de compenser le fait qu'il existe beaucoup plus de paires négatives que positives. L'optimisation est réalisée à l'aide de l'optimiseur ADAM avec un taux d'apprentissage de 0,001 et des batchs de 32 images. Enfin, le fait d'entraîner conjointement la détection et la description permet à ces deux tâches de s'aider mutuellement : savoir ce qui rend un point distinctif améliore sa localisation, et inversement, une meilleure localisation facilite l'apprentissage de descripteurs plus discriminants.

3) Première approche d'optimisation : La quantification INT8

Afin de commencer, j'ai décidé de m'attaquer à SuperPoint, qui me paraissait plus simple étant un CNN et car il constitue le point d'entrée du pipeline. Pour cela, j'ai testé la quantification. Une première quantification en INT8 a été réalisée, le but étant de diviser la taille du modèle par 4 (en passant de Float32 à INT8). De plus, les processeurs sont généralement plus adaptés à des modèles en INT8 ou FP16. Le but étant, à terme, d'embarquer le modèle optimisé, la quantification en INT8 semblait être un bon point de départ.

Malheureusement, j'ai rencontré plusieurs problèmes dus à cette méthode. En effet, il y avait un blocage lors de l'exportation vers ONNX ou TorchScript, avec des erreurs telles que TorchExportError ou GuardOnDataDependentSymNode. Ces erreurs étaient dues au fait que le nombre de points détectés varie d'une image à l'autre, ce qui crée un conflit avec les tenseurs qui, eux, sont statiques. De plus, la méthode torch.jit.trace a figé la logique interne du modèle quantifié en INT8, rendant le modèle incapable de se généraliser à d'autres dimensions d'images.

Pour résoudre ce problème, j'ai utilisé un wrapper manuel. Le but était de faire en sorte que seules les convolutions pures du backbone VGG et des décodeurs s'exécutent. Grâce à cela, un graphe ONNX 100 % statique a pu être exporté. Afin de limiter la perte de précision, une méthode de quantification statique avec calibration a été implémentée. Un DataLoader de calibration composé d'images réelles a été instancié pour faire converger des observateurs et figer proprement les plages dynamiques (*scales* et *zero-points*) des activations.

4) Protocole d'évaluation et métriques clés

Afin d'évaluer et de comparer les modèles pour ce projet, j'ai utilisé deux métriques : la répétabilité et le MLE.

La répétabilité correspond à la proportion de points de l'enseignant possédant au moins un point de l'élève situé à une distance inférieure ou égale à un rayon d'incertitude fixé $\epsilon = 3$ pixels. Elle se calcule selon la formule suivante :

$$\text{Répétabilité} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} \left(\min_j D_{i,j} \leq \epsilon \right) \times 100$$

Le MLE (Mean Localization Error) mesure l'erreur de localisation moyenne, exprimée en sous-pixels, calculée exclusivement sur l'ensemble des correspondances géométriques validées sous le seuil ϵ . Il se calcule selon la formule suivante :

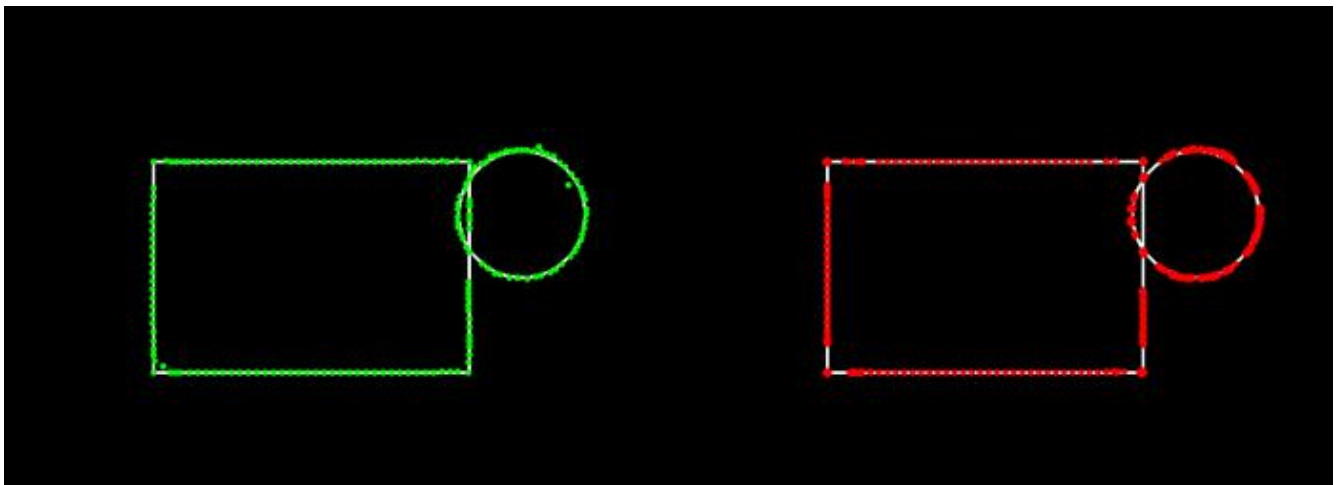
$$\text{MLE} = \frac{\sum_{i=1}^N \min_j D_{i,j} \cdot \mathbb{I}(\min_j D_{i,j} \leq \epsilon)}{\sum_{i=1}^N \mathbb{I}(\min_j D_{i,j} \leq \epsilon)}$$

5)Analyse des résultats de la quantification

INT8

Maintenant que nous disposons des métriques d'évaluation, nous pouvons analyser les résultats du modèle quantifié en INT8 :

- **Taille du modèle :** Au niveau de l'espace mémoire, nous sommes passés de 5 Mo pour le SuperPoint de base à 1,26 Mo pour le modèle quantifié. On constate donc bien la réduction par 4 de la taille du modèle, comme prévu.
- **Performances (MLE) :** Le résultat pour le MLE est très satisfaisant. Nous obtenons un score de 0,67, ce qui signifie qu'en moyenne, l'erreur de localisation n'est que de 0,67 pixel par rapport aux points détectés par le SuperPoint de base. L'erreur est donc particulièrement faible.
- **Performances (Répétabilité) :** Le bilan est en revanche plus mitigé pour la répétabilité. Même si l'on s'attendait naturellement à une perte de précision due à la quantification, le score s'effondre à 18,50 %. Cela signifie que seulement 18,50 % des points détectés sont identiques entre le modèle de base et le modèle quantifié.



Bien que les points soient très proches en moyenne (comme le montre le MLE), j'ai décidé de changer d'approche afin de tenter d'obtenir une meilleure répétabilité et de trouver un compromis idéal entre la réduction de la taille du modèle et le score des métriques.

6)Pruning et Distillation de connaissances

La stratégie a donc pivoté vers une approche combinant le pruning et la distillation de connaissances. Pour l'élagage, j'ai opté pour un pruning structurel. Cela signifie que je

n'ai supprimé aucune couche de convolution au sein de l'architecture, mais que j'ai plutôt choisi de réduire le nombre de filtres des couches de l'encodeur du CNN. Le modèle ainsi allégé est ensuite entraîné à imiter un modèle teacher qui est ici le SuperPoint de base.

L'hypothèse étant que certains canaux du CNN sont redondants ou moins importants que d'autres ; leur suppression devrait donc avoir un impact acceptable sur les performances au vu des gains obtenus en temps de calcul et en taille de modèle. Pour cette approche, nous choisissons de ne pas appliquer de quantification, ce qui signifie que le modèle conserve ses poids au format Float32 (FP32).

Concernant le choix du modèle student (l'élève, c'est-à-dire le modèle élagué que l'on va entraîner), nous allons tester deux configurations : un premier modèle avec un pruning de 50 % des canaux, et un second avec un pruning de 25 % des canaux.

Pour la configuration à 25 % par exemple, le nombre de filtres de chaque couche de l'encodeur est multiplié par un coefficient de 0.75. Ainsi, une couche de convolution qui possédait initialement 128 filtres n'en conserve plus que 96. Cette logique de réduction est appliquée uniformément sur l'ensemble des blocs de l'encodeur.

Bien que la capacité de l'encodeur soit réduite par le pruning, les têtes de description (convDa et convDb) sont conçues pour projeter les caractéristiques vers une dimension finale strictement maintenue à $D = 256$ canaux. Cette spécificité architecturale garantit que l'espace des descripteurs de l'élève conserve la même dimensionnalité structurelle que celle de l'enseignant. Cela permet d'éviter toute incompatibilité de format lors de l'intégration future du matcher (LightGlue).

Malheureusement, dès la mise en place de cette méthode, un problème est survenu : le modèle élève était « muet ». En d'autres termes, il ne détectait aucun point dans l'image et les heatmaps étaient composées à 98 % de pixels vides. Cela était dû au fait que la loss utilisée, la MSE (Mean Squared Error), menait le modèle élève à converger vers une solution triviale. Ainsi, il ne détectait rien, ce qui, mathématiquement, lui permettait pourtant de minimiser sa loss.

7) Conception de la fonction de perte (Loss) Hybride

Afin de résoudre ce nouveau problème, j'ai mis en place une Loss Hybride. La première composante de cette fonction est la suivante :

1. Perte de Détection (L_{det})

Au lieu d'une MSE brute, on applique une fonction Softmax sur les 65 canaux du tenseur de l'enseignant pour obtenir sa distribution de probabilités P_t . On calcule ensuite l'entropie croisée avec la distribution log-softmax de l'élève P_s .

La formule de cette perte de détection s'exprime ainsi :

$$\mathcal{L}_{det} = \frac{1}{B \cdot H_c \cdot W_c} \sum_{b=1}^B \sum_{i=1}^{H_c} \sum_{j=1}^{W_c} \sum_{c=1}^{65} -P_T(b, c, i, j) \cdot \log(P_S(b, c, i, j))$$

2. Perte de Description (L_{desc})

Pour contraindre l'alignement directionnel des descripteurs de l'élève sur l'espace latent du maître sans introduire de dérives géométriques, une perte basée sur la similarité cosinus moyenne est exploitée.

Elle se calcule de la manière suivante :

$$\mathcal{L}_{desc} = 1 - \frac{1}{B \cdot H_c \cdot W_c} \sum_{b=1}^B \sum_{i=1}^{H_c} \sum_{j=1}^{W_c} \text{CosineSimilarity}(D_S(b, :, i, j), D_T(b, :, i, j))$$

3. Perte Totale (L_{total})

La fonction de perte finale combine les deux composantes précédentes. Un coefficient de pondération agressif $h=20$ est appliqué pour accentuer l'importance de l'apprentissage des descripteurs.

La formule de la perte totale s'exprime ainsi :

$$\mathcal{L}_{total} = \mathcal{L}_{det} + 20 \cdot \mathcal{L}_{desc}$$

8) Analyse des performances des modèles prunés

Résultats pour le modèle pruné à 50 %

Pour le modèle pruné à 50 %, l'entraînement a été réalisé sur 15 époques, permettant de passer d'une loss de 6,5790 à 1,1968. Au-delà de 15 époques, la loss recommençait à remonter, ce qui indiquait un début d'*overfitting*.

Cependant, un problème subsistait : le modèle student affichait une confiance dans ses prédictions inférieure à celle du modèle de base (teacher). Pour corriger cela, j'ai recherché le seuil de détection optimal. Pour trouver le seuil idéal, j'ai utilisé un algorithme teste automatiquement une centaine de valeurs différentes sur l'image de test. Il compte combien de points le modèle student détecte à chaque essai et sélectionne précisément le seuil qui lui permet d'isoler la quantité exacte de points voulue. et a été fixé à 0,01169.

Après avoir appliqué ce seuil, j'ai évalué le modèle et obtenu les résultats suivants. Pour le nombre de points détectés, le modèle *Teacher* en trouve 200 contre 195 pour le modèle *Student*. La répétabilité atteint un score de 45,50 % et la précision (MLE) s'établit à 0,3648 pixel et avait une taille de 2.16Mo (donc une réduction de 56.80% de la taille).

Ces résultats montrent une nette amélioration par rapport à la quantification INT8. Le nombre de points détectés est presque identique à celui du modèle de base, l'erreur de localisation (MLE) reste extrêmement faible (en dessous du demi-pixel), et la répétabilité remonte de manière significative.

Résultats pour le modèle pruné à 25 %

Ces résultats nous montrent que la technique de pruning combinée à celle de la distillation possède un potentiel plus élevé que celle de la quantification. En effet, en observant le score du MLE, on constate que même lorsque le modèle student ne détecte pas exactement le même point que le modèle teacher, il n'est en réalité qu'à 0,3648 pixel en moyenne de celui-ci, tout en affichant une répétabilité bien plus élevée que le modèle quantifié en INT8.

Afin de déterminer s'il était possible d'obtenir de meilleurs résultats, et bien que les performances du modèle précédent soient déjà très satisfaisantes, j'ai voulu tester la même technique avec un pruning de 25 %, toujours en FP32. Pour ce dernier, j'ai réalisé un entraînement sur 18 époques et j'ai effectué deux tests distincts.

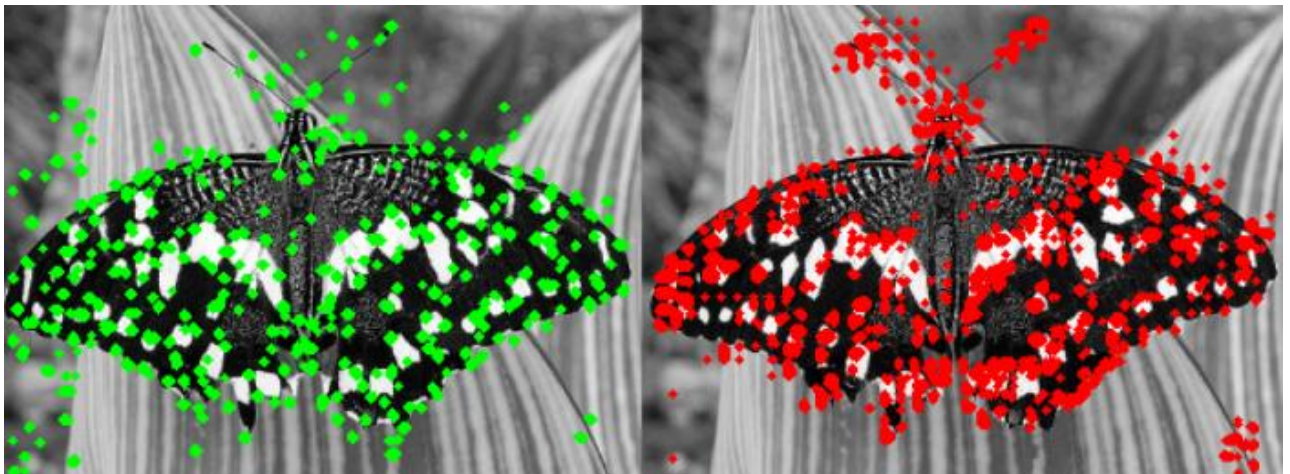
Premièrement, sur des formes géométriques simples telles que des carrés ou des triangles (comme pour le modèle en INT8, voir Figure 1), le modèle pruné à 25 % s'en est très bien sorti. En effet, il atteint un taux de répétabilité élevé de 82,66 % et une erreur de localisation moyenne (MLE) de seulement 0,6625 pixel avec une taille de 3.48Mo (une réduction de 30.40% de taille). Cela montre que la réduction de la taille du réseau

ne l'empêche pas de détecter et de placer précisément des formes géométriques nettes.

Ensuite, j'ai voulu tester le modèle sur une image plus complexe, qui se rapprocherait davantage d'une utilisation réelle du pipeline, afin de savoir s'il est capable de généraliser au-delà des formes simples. Il a donc été testé sur une image de papillon. Dans ces conditions réelles, le comportement du modèle évolue comme on pourrait s'y attendre :

Baisse de la répétabilité : Le taux de répétabilité tombe à 53,63 %. Cette baisse s'explique par le fait que les filtres de l'élève réagissent fortement aux détails des ailes du papillon, créant des points clés secondaires. Visuellement, le modèle reste pourtant efficace et détecte bien les contours principaux de l'insecte.

Maintien de la précision : Malgré la complexité de l'image, l'erreur de localisation reste très bien maîtrisée à 0,89 pixel. Ce score est largement en dessous du seuil critique de 3 pixels, ce qui prouve que le modèle reste stable.



Malgré le fait que le modèle pruné à 25 % affichait une répétabilité supérieure, le modèle le plus intéressant pour la suite me semblait être celui pruné à 50 %. Ce choix se justifie par sa taille plus réduite, mais également par son MLE très faible, qui montre que les points détectés par ce modèle sont presque identiques à ceux détectés par le SuperPoint de base.

TABLE 1 – Comparatif global des performances, tailles et réductions des modèles

Modèle	Format	Taille	Réduction	MLE	Répétabilité
Teacher (Original)	Float32	5,20 Mo	0,00 %	–	100 %
Student (<i>scale</i> = 0.75)	Float32	3,48 Mo	33,08 %	0,89 px	53,63 %
Student (<i>scale</i> = 0.50)	Float32	2,16 Mo	58,46 %	0,36 px	45,50 %
Modèle Quantifié	INT8	1,26 Mo	74,80 %	0,67 px	18,50 %

9) Intégration de LightGlue et analyse de l'espace latent

Une fois les modèles réduits de SuperPoint prêts, l'étape suivante consistait à poursuivre la construction du pipeline en y intégrant le *matcher* LightGlue. Pour ce faire, j'ai souhaité réaliser un premier essai avec le modèle pruné à 25 %. En effet, même si le modèle retenu pour le pipeline final reste celui pruné à 50 %, j'ai préféré utiliser pour ce premier test d'intégration le modèle le plus proche du SuperPoint original afin d'établir une *baseline* de comparaison fiable.

Malheureusement, j'ai là encore dû essuyer un échec. Alors que le pipeline de référence (SuperPoint d'origine + LightGlue) affichait 19 correspondances valides sur nos images réelles de test, l'intégration du modèle distillé a conduit à un résultat de 0 *match* valide trouvé. Pourtant, j'avais configuré le modèle student pour que la dimension de ses descripteurs reste strictement identique à l'original ($D = 256$), ce qui éliminait d'emblée toute possibilité d'erreur structurelle ou de format.

J'en ai donc conclu que le problème provenait de l'espace latent. En effet, bien que le modèle élève affiche une bonne répétabilité géométrique (prouvant qu'il localise visuellement les mêmes coins physiques), l'espace vectoriel de ses descripteurs a subi une rotation ou une déformation mathématique au cours de la distillation. Or, LightGlue a été entraîné pour attendre en entrée la distribution numérique exacte du SuperPoint original. Il se crée donc un conflit entre les descripteurs générés en sortie par le modèle student et ceux attendus en entrée par LightGlue.

Ce modèle student étant celui qui se rapprochait le plus du SuperPoint de base et l'intégration n'ayant pas fonctionné, j'ai pris la décision de ne pas tester les autres configurations (notamment la version à 50 %). Celles-ci auraient, au minimum, présenté les mêmes barrières techniques et les mêmes problèmes d'intégration.

10) Pivot méthodologique et contraintes matérielles

J'ai alors dû envisager un changement d'approche pour la suite du projet. En effet, ne pouvant pas simplement optimiser SuperPoint, puis optimiser LightGlue de manière isolée pour ensuite les assembler, il me fallait réfléchir à une meilleure option.

La première idée qui m'est venue à l'esprit a été de réentraîner le pipeline en end-to-end. L'objectif était d'entraîner à nouveau LightGlue directement sur les sorties de mon modèle student pruné à 25 %, afin qu'il apprenne à s'adapter à ce nouvel espace latent. Cependant, travaillant sur Google Colab avec un unique GPU T4, il semblait irréaliste de reprendre de zéro l'entraînement d'un transformer comme LightGlue. J'ai alors tenté de solliciter l'université pour utiliser leur cluster de calcul, mais je n'ai malheureusement jamais pu obtenir l'accès à cette ressource.

Face à ce revers technique et matériel, j'ai mené une analyse comparative des deux modèles principaux du pipeline (SuperPoint et LightGlue) afin de déterminer sur quel élément mon focus devait se porter pour mener à bien ce projet :

Analyse de l'empreinte mémoire du CNN (SuperPoint)

En sachant que le pipeline total pèse 125 Mo, l'empreinte de SuperPoint est particulièrement réduite. Il repose sur une architecture de type CNN classique et compacte de seulement 5 Mo. À l'échelle d'un système embarqué moderne, un poids fixe de 5 Mo représente une charge mémoire totalement négligeable, puisqu'il ne constitue que 4 % du poids total du pipeline initial.

L'impact du Transformer (LightGlue)

En comparaison, le modèle LightGlue original complet à 9 couches pèse environ 120 Mo. Il est composé de blocs d'attention Transformers lourds qui génèrent une complexité quadratique saturant la RAM et l'unité de calcul.

Ce constat a imposé un pivot logique immédiat : puisque l'intégration directe de notre élève le plus proche a échoué et que la taille de SuperPoint est totalement négligeable par rapport à celle de LightGlue, la stratégie optimale à ce stade consiste à geler temporairement SuperPoint dans sa version originale stable de 5 Mo. Ce gel garantit une compatibilité absolue avec le pipeline officiel et nous permet de concentrer l'intégralité de notre intelligence d'optimisation embarquée sur le Transformer LightGlue, là où les gains potentiels sont bien supérieurs.

11) Explication de LightGlue

Tout d'abord, avant LightGlue, le state of the art était SuperGlue, dont le but était d'analyser deux images en même temps afin de mettre en correspondance des points d'intérêt et de rejeter les outliers. SuperGlue est basé sur une architecture Transformer et utilise SuperPoint : ce dernier fournit les localisations et les descripteurs des points d'intérêt, tandis que SuperGlue a pour rôle de comparer ces points et de trouver les correspondances entre les deux images. Cependant, SuperGlue présente plusieurs problèmes. Tout d'abord, il est très lent et également très coûteux à entraîner, nécessitant des ressources de calcul importantes (plusieurs jours, voire semaines de GPU). De plus, sa complexité est quadratique en fonction du nombre de points d'intérêt. Il ne possède aucune adaptabilité : quelle que soit la difficulté de la paire d'images, il utilisera toujours la même quantité de calcul. Enfin, il repose sur l'algorithme de Sinkhorn, qui permet de transformer une matrice de similarité en une matrice d'assignation où chaque point de l'image A est associé de manière unique (ou non) à un point de l'image B. Cet algorithme est cependant très gourmand en mémoire et ne permet pas un entraînement efficace couche par couche. C'est pour l'ensemble de ces raisons que LightGlue a été proposé. LightGlue est un réseau de neurones profond basé sur une architecture de Transformers, ce qui permet un apprentissage conjoint puisque le modèle traite les deux images simultanément. Son objectif est d'être une amélioration directe de SuperGlue, à la fois plus rapide, plus efficace et plus simple à entraîner. LightGlue est composé de 9 couches d'attention identiques, chacune contenant un module de self-attention, dont le but est que les points d'une même image communiquent entre eux afin de comprendre le contexte global, ainsi qu'un module de cross-attention bidirectionnel, permettant aux points de l'image A et de l'image B de se comparer et de rechercher leurs correspondances. En tant que Transformer, LightGlue utilise le Rotary Positional Embedding (RoPE). Contrairement aux encodages positionnels absolus, il n'utilise pas directement les positions exactes des points, mais leurs positions relatives les uns par rapport aux autres, via des rotations. En effet, lorsqu'une caméra se déplace, les points changent de position absolue dans l'image, mais leurs relations géométriques relatives restent cohérentes. Après les mécanismes d'attention, un MLP (un FFN) permet de mettre à jour l'état de chaque point (son vecteur de caractéristiques) pour la couche suivante. Là où LightGlue commence réellement à se démarquer de SuperGlue, c'est qu'immédiatement après le MLP, une Prediction Head est appliquée. Celle-ci calcule deux éléments : une matrice d'assignation P (quel point va avec quel point) en multipliant les descripteurs de l'image A par ceux de l'image B et un score de confiance en utilisant dual-softmax. À partir de ce score de confiance, LightGlue introduit un critère d'arrêt (early exit). Si le score de confiance global dépasse un seuil prédéfini (généralement compris entre 0.95 et 0.99),

le modèle s'arrête immédiatement. Ainsi, pour des images très similaires, les scores de confiance peuvent être élevés dès les premières couches, ce qui permet d'éviter des calculs inutiles. Si ce critère n'est pas atteint, le modèle applique alors un pruning qui utilise quant à lui la sigmoïde. C'est-à-dire que si la probabilité associée à un point descend en dessous d'un seuil fixé (souvent très faible, par exemple 0.01, afin de ne pas supprimer des points utiles), ce point est supprimé des calculs suivants. Plus on avance dans les couches, moins il reste de points, ce qui réduit fortement la quantité de calcul nécessaire car l'attention du Transformer ayant une complexité de $O(N^2)$ si l'on supprime par exemple la moitié des points on divise le nombre de calcul par 4. À la fin des 9 couches, ou après un early exit, le modèle renvoie, pour chaque point détecté dans l'image A, l'index du point correspondant dans l'image B ainsi que le score de confiance associé. Il identifie également les points ne possédant aucune correspondance. Concernant l'entraînement, LightGlue est entraîné sur de très grandes bases de données, telles que MegaDepth, pour lesquelles on connaît précisément les correspondances géométriques entre images. L'entraînement repose sur la Deep Supervision, c'est-à-dire que l'on calcule une loss après chaque couche. Cela rend l'entraînement plus stable et plus rapide, et c'est ce mécanisme qui permet au pruning et à l'early exit d'être aussi efficaces. Pour chaque couche, une Binary Cross-Entropy (BCE) est utilisée comme fonction de perte, et la loss totale correspond simplement à la moyenne pondérée des loss calculées à chaque couche. C'est l'ensemble de ce processus qui permet à LightGlue d'être entraîné en seulement quelques jours, là où SuperGlue pouvait nécessiter plusieurs semaines de calcul GPU.

12) Optimisations LightGlue

Optimisation par compilation Just-In-Time (JIT)

Afin d'optimiser LightGlue, j'ai dans un premier temps utilisé la compilation Just-In-Time (JIT) de PyTorch. Pour rappel, la compilation JIT transforme un modèle écrit en code Python en un format figé, optimisé et ultra-rapide. Elle permet notamment de détacher le modèle de l'interpréteur Python pour pouvoir l'exécuter directement en C++ sur des serveurs de production ou des applications mobiles.

Grâce à cette méthode, j'ai obtenu un gain de 20,6 % de FPS (FPS = La quantité d'images affichées en une seconde), atteignant ainsi 36,2 FPS et une latence de 27,6 ms (Latence (ms) = Le temps d'attente pour traiter une information), là où le modèle LightGlue original tournait à 30 FPS et avait une latence de 27.63ms. Cette accélération s'est faite tout en conservant une fidélité géométrique stricte de 100 %.

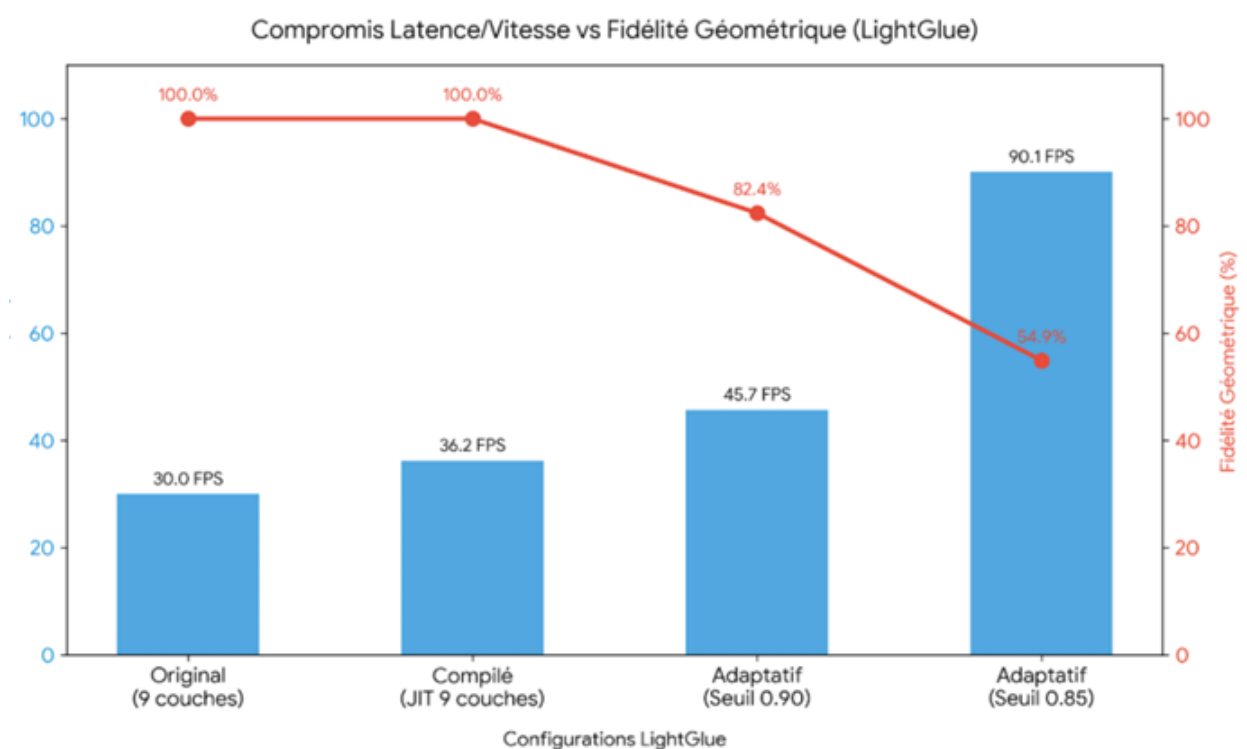
Optimisation par seuil d'arrêt précoce (*Early Stopping*)

Ensuite, l'optimisation que j'ai trouvée la plus intéressante à tester consistait à jouer sur le seuil d'early stopping. En effet, en appliquant un seuil plus agressif, le modèle utilise en moyenne moins de couches transformer pour valider ses correspondances.

Le premier essai a été réalisé avec un seuil fixé à 0,90. Les résultats ont été particulièrement encourageants et convaincants : j'ai obtenu une vitesse de 45,7 FPS et une latence réduite à 21,9 ms, tout en maintenant une répétabilité géométrique très satisfaisante de 82,4 %.

À la suite de ces résultats, j'ai voulu déterminer s'il était possible de pousser l'optimisation encore plus loin en fixant le seuil à 0,85. Cependant, ce dernier s'est révélé un peu trop agressif. En effet, même s'il a permis d'atteindre une vitesse de 90,1 FPS (soit une latence très faible de 11,0 ms), ce qui représente un gain extrêmement important par rapport au LightGlue de base, il a provoqué un effondrement de la fidélité géométrique, qui est tombée à 54,9 %.

Au vu de ces résultats, le modèle le plus intéressant m'a paru être celui configuré avec un seuil de 0,90. Il offre selon moi le meilleur compromis, en conservant un équilibre acceptable entre gain de fluidité et précision géométrique.



13) Conclusion et perspectives d'avenir

Ces derniers tests clôturent mes expérimentations sur le pipeline combinant SuperPoint et LightGlue. Bien que j'aurais souhaité mener d'autres investigations, les restrictions liées à la puissance de calcul disponible et aux contraintes de temps ont limité la portée de mes essais.

Néanmoins, plusieurs perspectives prometteuses s'ouvrent pour la suite de ce projet, à condition de disposer d'une infrastructure de calcul plus importante. L'axe de recherche principal reposerait sur la distillation de connaissances appliquée directement à LightGlue. En effet, les auteurs du papier de recherche initial démontrent qu'au bout de seulement 5 couches de *Transformer*, LightGlue parvient déjà à identifier 90 % des correspondances finales. Il serait donc particulièrement pertinent de concevoir une stratégie d'optimisation incrémentale en trois étapes :

- **Étape 1 : Distillation structurelle de LightGlue (Modèle à 5 couches)** Dans un premier temps, l'objectif serait de créer un modèle *student* de LightGlue limité à 5 couches de *Transformer*. Pour ce faire, il faudrait impérativement conserver le modèle SuperPoint original gelé. Cette approche permettrait d'isoler les variables, de conserver une *baseline* de comparaison fiable et de valider si cette réduction structurelle est viable.
- **Étape 2 : Réalignement de l'espace latent** Dans un second temps, il conviendrait de réaliser la démarche inverse : réentraîner le modèle LightGlue complet (à 9 couches), mais en lui fournissant en entrée les sorties (*outputs*) du modèle SuperPoint *student* pruné à 50 %. Cela permettrait à LightGlue d'apprendre à interpréter et à corriger les déformations de ce nouvel espace latent.
- **Étape 3 : Co-distillation optimale du pipeline** Enfin, si les résultats de ces deux expériences s'avéraient satisfaisants, l'aboutissement de cette recherche consisterait à fusionner les deux bénéfices. On procéderait alors à la distillation d'un nouveau LightGlue à 5 couches, entraîné sur les sorties du SuperPoint pruné à 50 %, en utilisant comme modèle *teacher* le LightGlue complet fraîchement réentraîné à l'étape précédente.

Une telle approche de co-distillation permettrait d'obtenir un pipeline global bien plus léger, rapide et parfaitement aligné, maximisant ainsi les performances pour un déploiement sur système embarqué.

Conclusion

Bien qu'il soit difficile d'anticiper avec certitude les résultats de ces nouvelles pistes d'expérimentation, la taille globale du pipeline passerait alors de 125 Mo à un peu plus de 62 Mo. Cette réduction de moitié représenterait d'ores et déjà un véritable succès pour l'intégration et le déploiement de ce pipeline au sein de systèmes embarqués aux ressources limitées.

